Autonomous Pong Paddle Receiver: Real-Time Vision, Intercept Prediction, and Closed-Loop Paddle Actuation on a Belt-Driven Linear Rail

Maximilian Strey
Dept. of Electrical Engineering
The Cooper Union
New York, NY, USA
maximilian.strey@cooper.edu

***Abstract*—This work develops an autonomous receiver capable of intercepting a thrown ping-pong ball using only a single overhead camera, a laptop, and a belt-driven linear rail. The system was originally proposed as a spin-aware return mechanism employing a Core-XY traverse and a two-axis paddle gimbal; this scope was reduced after measurement showed the available camera could not resolve spin within its frame budget. The delivered system implements the full perception–prediction–actuation loop on a single lateral axis: a two-path vision detector (circular blob plus motion streak) finds the ball in each frame, a rolling least-squares fit estimates the ball's velocity vector, and a bounce-aware geometric extrapolation projects the trajectory onto the rail. A three-state machine gates motor commands so that post-bounce noise does not whip the paddle, and a hierarchical motion-scaling rule prevents belt-skip events under aggressive acceleration. The system intercepts approximately two of every three direct shots at moderate incoming speeds, dropping to roughly one in two when the trajectory involves a wall bounce.**

***Index Terms*—real-time vision, object tracking, trajectory prediction, stepper motor control, embedded systems, sports robotics**

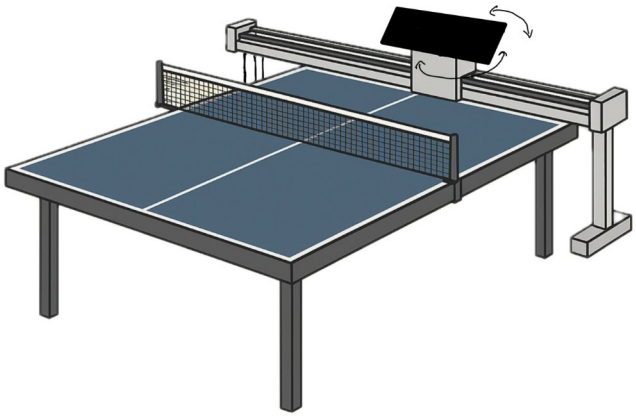

Fig. 1. Original system concept from the project proposal: a Core-XY gantry mounted behind the far end of a regulation table, with a two-axis tilt paddle. The delivered system implements the lateral axis only, on a shorter 3 ft rail (Fig. 2); the descope is discussed in Section VII.

I. Introduction

Table tennis training relies on consistent, repetitive rallies. A player practicing alone has limited tools: folding the table half upward produces inconsistent rebound angles, and commercial ball launchers [1], [2] feed balls without reacting to the player's return. A truly useful solo trainer must *receive*—it must perceive a ball in flight and physically intercept it. This is a compact problem in real-time perception, trajectory prediction, and closed-loop motion control: only a few hundred milliseconds elapse between when a ball becomes visible and when it must be met.

This project was originally proposed as a full autonomous receiver (Fig. 1): a Core-XY traverse moving a paddle across the far end of the table, a two-axis gimbal tilting the paddle to counter top- and back-spin, and two cameras (overhead and side) inferring spin class from bounce geometry. The architecture was inspired by Omron's Forpheus [3] and the DeepMind robotic table tennis work [4], whose use of a 2-DOF linear gantry behind a robotic arm motivated the original mechanical plan of decoupling lateral coverage from paddle articulation.

During implementation, we discovered that our camera could not measure spin within its useful frame budget (§VII). Rather than deliver a non-functional spin-aware system, the scope was narrowed to a single lateral axis on a shorter rail: a "pong-style" receiver that demonstrates the full perception–prediction–actuation loop with honest hardware that actually works. The technical sections below describe the delivered system; the descope and what was learned from it are discussed explicitly in Section VII.

Authorship: The project was originally proposed with Isaac Amar, who co-developed the initial planning and contributed an early vision-code prototype implementing motion-and-brightness gated circular blob detection. After the fall

2025 semester, Isaac disengaged from the project; all work described from Section IV onward, beyond the original circular-blob detector, was completed solo by the author.

II. Background and Related Work

A. Robotic table tennis

Omron's Forpheus [3] demonstrates that a multi-axis robotic arm with high-speed vision can sustain cooperative rallies with a human partner, but its scale, sensor cost, and tightly integrated control stack place it well outside an undergraduate educational budget. The Google DeepMind table tennis system [4] combines an industrial 6-DOF arm with a 2-DOF Festo linear gantry, using learning-based control to sustain extended rallies with a human. Both systems served as conceptual references for this project rather than as direct technical predecessors; in particular, the idea of using a linear gantry to extend the reach of an articulated end effector motivated the original Core-XY plan and is revisited as future work.

B. Vision tracking

Classical ball tracking pipelines typically combine background subtraction or frame differencing with a shape-based filter such as the Hough circle transform or a circularity check on contour geometry. We use a frame-differencing motion mask combined with an adaptive brightness threshold, implemented in OpenCV [5], with two parallel scoring criteria to handle both static and motion-blurred ball appearances. Local contrast enhancement uses CLAHE (Contrast Limited Adaptive Histogram Equalization) [6], which normalizes brightness on a per-tile basis to make the brightness threshold robust to uneven illumination.

C. Stepper motor control

The Arduino UNO R4 firmware uses AccelStepper [7] to generate the step pulses for a DM542 microstep driver. AccelStepper plans trapezoidal velocity profiles: the motor accelerates at a configured rate until reaching a configured peak speed, cruises, then decelerates symmetrically. For a move shorter than the distance needed to reach peak speed, the profile collapses to a triangle. We exploit this property to set acceleration bounds that prevent the belt from skipping teeth under short, aggressive commands (§VI-B).

III. System Architecture

The system has three computational components and a single mechanical axis. Frames from an overhead global-shutter camera arrive at a laptop, which runs the vision pipeline and trajectory predictor. The laptop streams target step positions over USB serial to an Arduino, which generates the step pulses for a microstep driver. The driver energizes the windings of a NEMA 17 stepper, which turns a 20-tooth pulley and drives a GT2 belt; a carriage on the belt rides along an aluminum-extrusion rail.

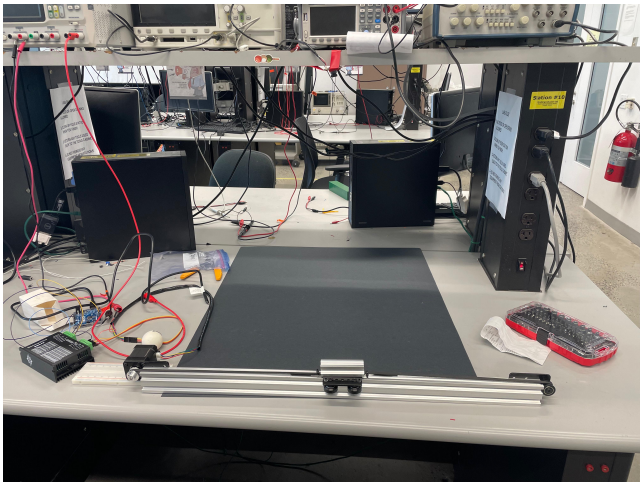

Fig. 2. Delivered hardware on the lab bench: belt-driven NEMA 17 stage on a 3 ft aluminum extrusion, with the DM542 microstep driver, breadboard, and Arduino visible to the left. The paddle mounts to the carriage at the center of the rail; the camera (held by red tape) mounts overhead.

A. Hardware

- **Camera.** Arducam B0332 with the OV9281 sensor [9], a monochrome global-shutter imager. Native output is 8-bit grayscale at 1280×800 resolution, up to 120 fps over USB. Global shutter is essential here: a rolling shutter would smear a fast-moving ball into an unrecognizable streak with spatially varying timing.
- **Compute.** A consumer laptop runs vision and prediction in Python with OpenCV [5]. The use of OpenCV compiled with vectorized backends keeps per-frame processing time below the 16 ms frame budget at 60 fps.
- **Microcontroller.** Arduino UNO R4 Minima [8] running AccelStepper [7] over USB serial at 115,200 baud. The microcontroller exposes a non-blocking position server: it accepts new step targets at any time without interrupting an in-progress motion, recomputes the trapezoidal profile from current state, and answers position queries while moving.
- **Actuator.** NEMA 17 stepper, DM542 microstep driver configured for 800 pulses/rev, GT2 belt on a 20-tooth pulley running on a 3 ft aluminum-extrusion rail.

B. Why split compute?

A laptop running Python under a multitasking operating system cannot reliably issue step pulses with the sub-millisecond timing precision required by a stepper driver. A scheduler hiccup of 20–50 ms would cause the motor to lose steps and silently miscalibrate the system. The Arduino's role is to absorb this timing requirement: the laptop emits high-level target positions, and the Arduino converts them into the deterministic pulse trains that the driver expects.

IV. Vision Pipeline

The vision pipeline runs once per frame and produces, when possible, a single tuple: the pixel coordinates and timestamp of

the detected ball. The pipeline operates entirely within a user-defined rectangular region of interest (ROI) corresponding to the physical play area; the ROI is established once per session during calibration (§IV-D).

A. Cropping and contrast equalization

Each incoming grayscale frame is cropped to the ROI. The cropped region is then processed by CLAHE [6] with an 8×8 tile grid and clip limit 2.0. CLAHE redistributes brightness on a per-tile basis with bounded contrast amplification, which prevents global brightness gradients (lamp on one side, shadow on the other) from forcing a single brightness threshold to either miss the ball on the dim side or trigger on the lit-up surface on the bright side.

B. Motion and brightness gating

Two binary masks are computed and combined. The *motion mask* flags pixels whose absolute difference from the previous equalized frame exceeds 18 (out of 255), after a 5×5 Gaussian blur to suppress sensor noise. The *brightness mask* flags pixels above $\mu + 1.5\sigma$, where μ and σ are the mean and standard deviation of the current equalized ROI, clamped to $[60, 240]$.

The masks are combined by elementwise AND. A pixel survives only if it moved *and* is brighter than typical. This filter pair eliminates two whole categories of false positive simultaneously: static bright objects (table edges, glare spots, tape) fail the motion test; moving dark objects (a hand, a shadow) fail the brightness test. Only a moving white ball survives both.

C. Two-path contour scoring

The combined mask is cleaned with morphological open-then-close operations and passed to OpenCV's contour finder. Each contour is scored under two independent criteria, run in parallel:

a) Slow-ball path: Computes circularity

$$C = \frac{4\pi A}{P^2} \tag{1}$$

where A is the contour's area and P its perimeter. A perfect circle has $C = 1$; a long thin streak has C close to zero. A contour is accepted on this path if $C > 0.45$, and scored by $C^2 \cdot A$.

b) Fast-ball path: Fits the minimum-area rotated bounding rectangle. The contour is accepted if its aspect ratio (long side / short side) exceeds 1.5 and the short side is between 3 and 40 pixels. Accepted contours are scored by their solidity, defined as the contour's area divided by the area of its convex hull.

The highest-scoring contour across both paths wins, and its centroid is reported as the ball's position. The two-path structure is motivated by motion blur: at the camera's typical 8 ms exposure, a ball moving at $v \approx 0.9$ m/s travels approximately 7 mm during the shutter window, producing a streak whose aspect ratio exceeds the boundary where a single circularity threshold can reliably catch it. Either threshold tuned for slow balls misses fast balls (or vice versa); running both criteria handles both regimes without retuning.

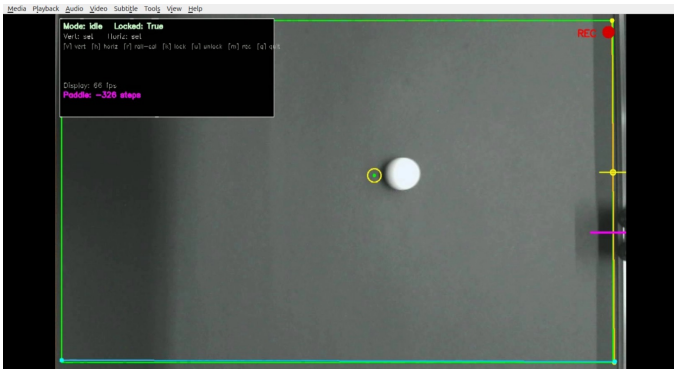

Fig. 3. Live tracker view during operation. Green: locked play-area rectangle. Yellow ring: detected ball centroid. Magenta tick on right edge: paddle position on the rail. Status overlay shows the state machine (§VI-A). The screenshot is captured during a session when the system was in the IDLE state between throws.

D. In-frame calibration

Rather than measuring rail dimensions with a ruler, calibration is performed entirely within the camera image. The user clicks two points to define a vertical rail axis and two more for the horizontal extent of the play area, producing the ROI. The user then jogs the carriage to the top and bottom of the rail's reachable range using keyboard arrows and presses a key at each extreme, recording the step count at that position. The system computes

$$\text{steps\_per\_pixel} = \frac{|s_{\text{bot}} - s_{\text{top}}|}{|y_{\text{bot}} - y_{\text{top}}|} \tag{2}$$

directly from the four observations: two step counts and two pixel positions. No physical measurement is required, and any errors from pulley geometry, belt stretch, or non-ideal mounting are absorbed into a single empirically-fit constant.

V. Trajectory Prediction

The predictor consumes the position–timestamp stream from the vision pipeline and produces, at each frame, an estimate of where the ball will reach the rail line $x = x_{\text{rail}}$.

A. Rolling-window velocity estimate

The most recent six (x, y, t) observations are held in a sliding window. A least-squares line is fit to the points in each axis independently, yielding slope estimates

$$\hat{v}_x = \frac{\sum_i (t_i - \bar{t})(x_i - \bar{x})}{\sum_i (t_i - \bar{t})^2}, \tag{3}$$

$$\hat{v}_y = \frac{\sum_i (t_i - \bar{t})(y_i - \bar{y})}{\sum_i (t_i - \bar{t})^2}. \tag{4}$$

Compared with a two-point finite difference, the six-point linear fit reduces per-frame measurement noise on the velocity estimate by roughly a factor of $\sqrt{6}$ under the assumption of uncorrelated noise in the detection centroids.

B. Bounce-aware geometric extrapolation

Once a valid velocity is available, the predictor computes where the trajectory will reach $x = x_{\text{rail}}$. In the absence of bounces, the time to intercept and the predicted y coordinate are

$$\tau = \frac{x_{\text{rail}} - x_0}{\hat{v}_x}, \tag{5}$$

$$y_{\text{pred}} = y_0 + \hat{v}_y \cdot \tau. \tag{6}$$

The lateral demo is to be installed inside a rectangular play area bounded by physical rods on the top and bottom edges. If the straight-line extrapolation in Eq. (6) would cross the top wall ($y < y_{\text{top}}$) or bottom wall ($y > y_{\text{bot}}$) before reaching the rail, the trajectory is recursively reflected at that wall, with $\hat{v}_y \to -\hat{v}_y$, and the calculation continues from the bounce point until the rail is reached or a safety limit on bounce count is hit. Speed magnitude is preserved through the bounce (perfectly elastic model). Position error from the elastic assumption is small for the demo's geometry because the rail intercept depends only on bounce *angles*, not on bounce *timing*; energy loss during bouncing affects the latter but not the former.

C. Exponential smoothing

Predictions on consecutive frames are smoothed by a one-pole exponential filter:

$$\bar{y}_{\text{pred}}^{(k)} = \alpha\, y_{\text{pred}}^{(k)} + (1 - \alpha)\, \bar{y}_{\text{pred}}^{(k-1)}, \tag{7}$$

with $\alpha = 0.55$. The filter trades off responsiveness against robustness to single-frame detection noise; the chosen value is biased toward responsiveness so the system has a useful early prediction within two or three frames of a fresh trajectory.

VI. Motion Control

The predictor produces a target $\bar{y}_{\text{pred}}$ on every frame, but motor commands are not issued on every frame. Two layers of gating decide whether and how to actuate.

A. Three-state machine

The Director runs a three-state machine:

- **IDLE.** No ball seen recently. Predictor history is flushed. No motor commands are sent.
- **TRACKING.** A valid forward-moving trajectory has been detected. The predicted intercept is updated each frame and the paddle is commanded toward the current estimate.
- **COMMITTED.** The ball has passed a configurable *commit line* at 60% of the play area's x-span. From this point until the trajectory ends, the target is frozen and no further motor commands are issued, no matter what the predictor reports.

The transition from TRACKING to COMMITTED solves a class of failure modes that single-shot prediction does not: a ball that bounces off the paddle reverses its x-velocity and generates fresh "predictions" in the wrong direction; small detections from the operator's hand crossing the ROI create noisy targets that whip the motor; and detection noise at high paddle speeds amplifies into target jitter. Freezing the target inside the commit zone makes the final approach insensitive to all of these.

The end-of-trajectory condition is a 300 ms gap with no successful detection, after which the machine returns to IDLE and prepares for the next throw. A spacebar reset is also available for manual recovery from stuck states during testing.

B. Hierarchical motion scaling

When TRACKING wants to issue a command, the Director must choose speed and acceleration parameters. A naive constant-envelope policy fails: high acceleration on a short move causes the belt to skip teeth (silently miscalibrating the rail), while low acceleration on a long move causes the paddle to arrive after the ball. The fix is to scale both parameters by the actual physical distance the paddle has left to travel:

$$d = |s_{\text{target}} - s_{\text{current}}|, \tag{8}$$

where s_{current} is the latest paddle position polled from the Arduino. Three regimes apply:

- $d \geq 400$ steps: full speed (4000 steps/s) and full acceleration (8000 steps/s^2).
- $80 < d < 400$ steps: linearly interpolated speed and acceleration.
- $d \leq 80$ steps: *dead band*—no command issued. The paddle is already close enough; AccelStepper cannot usefully plan such a short move, and re-commanding mid-ramp would interfere with the existing motion profile.

A final safeguard rescales acceleration when needed to keep peak velocity bounded. For a triangular motion profile (typical of short moves) the peak velocity is

$$v_{\text{peak}} = \sqrt{a \cdot d}. \tag{9}$$

If the chosen (a, d) would produce v_{peak} exceeding the chosen maximum speed, a is reduced so that v_{peak} equals the speed cap exactly. This keeps short moves smooth and prevents the motor from whipping in a way that the belt cannot follow.

The decision to scale by *physical remaining distance*—rather than by prediction change between consecutive frames—is a non-obvious but consequential one. An earlier version of the controller scaled by prediction Δy, which had the pathological behavior that a long move with a small frame-to-frame prediction refinement would drop the motor into low-speed mode while it was still mid-way through a large physical move. Scaling by d in Eq. (8) aligns the speed envelope with what the motor actually has to do.

VII. Scope Evolution: The Spin-Measurement Problem

The original proposal called for spin-aware return: the paddle would tilt to counter top- and back-spin (forward axis) and angle along the table for diagonal shots (vertical axis). This required estimating the ball's angular velocity from video, which in turn required resolving the printed marking

on the ball over enough consecutive frames to fit an angular trajectory.

In practice, the live camera feed yielded approximately nine usable ball-detection frames over a typical shot of $\sim$1 second duration in the play area. The remainder of the frames either lacked detection (ball outside the active ROI, motion-blurred below the detector's geometric thresholds) or contained the ball as an elongated streak rather than as a resolved circle in which the printed marking would be visible.

Nine detections is not enough to fit the angular velocity of a spinning ball with any reliability—and even those detections show the ball motion-blurred during the camera's exposure window, which smears the printed marking across an arc on the ball's surface rather than imaging it crisply. Resolving the spin would require either a substantially higher frame rate, a substantially shorter exposure, or a dedicated high-speed camera. None of these were available within the project's hardware budget.

We responded by removing the gimbal, the spin estimator, the side camera, and the depth (back–forward) axis from scope. The lateral axis, vision pipeline, predictor, and motor controller—the entire real-time loop—remain. The result is a system that solves a smaller version of the original problem well, rather than a larger version of it poorly.

VIII. Results and Evaluation

Systematic quantitative measurement was not performed within the project timeline. The results reported here are based on repeated testing during development and during a final integration session; they should be read as informed qualitative observations rather than as statistically calibrated metrics.

A. Operating regime

The system operates reliably on hand-thrown shots that traverse the $\sim$3 ft horizontal extent of the play area in approximately 1 second, giving an estimated mean ball speed of 0.9 ± 0.2 m/s. At this speed the ball appears as a partially-streaked blob whose aspect ratio sits near the boundary between the slow-ball and fast-ball detection paths of §IV-C, which is to say: both detection paths contribute usefully.

Substantially slower shots (which arc and curve under gravity and table-tilt effects, breaking the linear-velocity assumption of Eq. (3)–(4)) and substantially faster shots (for which the prediction window is too short to converge before the ball arrives) both fail. The working envelope is narrow but present.

B. Interception success rate

Within the operating regime, we observe:

- Approximately two-thirds of direct shots (no wall bounce involved) are successfully intercepted by the paddle.
- Approximately one-half of shots that involve a bounce off the top or bottom rod are successfully intercepted.

The drop in success rate when bounces are involved is consistent with the bounce-prediction model: real wooden rods are not perfectly elastic, so the post-bounce trajectory deviates from geometric prediction by a small angle that compounds with the remaining flight distance.

The originally-proposed full system included an actual paddle face with several centimeters of width tolerance for an intercept; the delivered demo uses a narrow marker on the carriage, so successful "interceptions" correspond to the marker meeting the ball within $\sim$1 cm of the predicted intercept line. The reported success rates would improve under the original paddle geometry.

C. Failure modes

Three failure modes dominate:

1) **Late prediction.** For faster shots, the predictor does not have enough samples in the early TRACKING phase to produce a converged intercept estimate before the commit line. The paddle commits to an early, noisy target.
2) **Belt skip.** Aggressive acceleration on long moves occasionally causes the GT2 belt to skip teeth on the drive pulley, silently shifting the rail's calibration by one or more tooth pitches. The motion-scaling rule of Eq. (8)–(9) reduced but did not eliminate this.
3) **Bounce-angle drift.** The perfectly-elastic reflection model in the bounce predictor is an upper bound on accuracy; lossy rod-ball contacts produce a small angular bias that the system cannot correct for.

IX. Ethical Considerations

The delivered system is a benchtop research demonstrator, not a deployed consumer product. Two safety considerations bear noting. First, the belt-driven carriage has pinch points at the drive pulley and idler that could injure fingers under operation; the demo should be operated with the rail's moving region clear of hands and loose materials. Second, the stepper-driver power supply is capable of delivering substantial current; standard benchtop electrical-safety practice (fused supply, grounded chassis, isolated logic) applies. The system has no autonomous operation modes: it begins motion only after explicit operator configuration and lock-in, and the operator may halt motion at any time via keyboard interrupt.

X. Conclusion and Future Work

The delivered system demonstrates the full perception–prediction– actuation loop for autonomous ball interception on real hardware, within a substantially reduced scope relative to the original proposal. The descope was motivated by a measurement, not a preference: spin estimation is below the optical limit of the available camera. The retained loop—two-path vision, rolling- window velocity estimation, bounce-aware geometric prediction, three-state command gating, and physical-distance-scaled motion— forms a working subset that could be extended back toward the original goal without redesign of its constituent parts.

The clearest path to the original spin-aware system is sensor upgrade. A high-speed camera (target: ≥ 1000 fps with sub-millisecond exposure) would resolve the ball's printed marking

across many frames per shot, enabling angular-velocity estimation and re-opening the gimbal-based spin-aware return. With spin estimation in hand, a two-axis paddle gimbal could close the return-angle loop. Beyond spin, a two-axis linear gantry of the kind used in [4] would add a depth axis to this system, allowing the receiver to handle balls whose trajectory carries them beyond the rail's reach.

REFERENCES

[1] Butterfly, "Amicus Prime table tennis robot," product page, Tamasu Butterfly Europa GmbH, 2025. [Online]. Available: https://en.butterfly.tt/b-amicus-prime.html

[2] Newgy Industries, "Robo-Pong table tennis robots," product documentation, 2025. [Online]. Available: https://www.newgy.com/collections/robots

[3] Omron Corporation, "FORPHEUS," technology page, 2025. [Online]. Available: https://www.omron.com/global/en/technology/forpheus/

[4] D. B. D'Ambrosio *et al.*, "Robotic table tennis: A case study into a high-speed learning system," in *Proc. Robotics: Science and Systems (RSS)*, 2023. https://doi.org/10.15607/RSS.2023.XIX.006

[5] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, Nov. 2000.

[6] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*, P. S. Heckbert, Ed. San Diego, CA: Academic Press, 1994, pp. 474–485.

[7] M. McCauley, "AccelStepper: Arduino library for stepper motor control with acceleration and deceleration," library documentation, 2025. [Online]. Available: https://www.airspayce.com/mikem/arduino/AccelStepper/

[8] Arduino, "UNO R4 Minima," hardware documentation, 2025. [Online]. Available: https://docs.arduino.cc/hardware/uno-r4-minima/

[9] OMNIVISION, "OV9281: 1/4-inch B&W 1-megapixel (1280×800) CMOS image sensor with OmniPixel®3-GS technology," product page, 2025. [Online]. Available: https://www.ovt.com/products/ov9281/